Python & Algorithms for Coding Interviews

Section 6 — Time Complexity Cheat Sheet

6.1 Data Structure Operations

Data Structure	Operation	Average	Worst	Space
Array / List	Access by index	O(1)	O(1)	O(n)
	Append (end)	O(1) amort	O(n)	
	Insert / Delete (mid)	O(n)	O(n)	
	Search (unsorted)	O(n)	O(n)	
Hash Map / Set	Get / Set / Delete	O(1)	O(n)	O(n)
	Membership (in)	O(1)	O(n)	
Stack (list)	Push / Pop / Peek	O(1)	O(1)	O(n)
Queue (deque)	Enqueue / Dequeue	O(1)	O(1)	O(n)
Heap (heapq)	Push / Pop	O(log n)	O(log n)	O(n)
	Peek min / Heapify	O(1) / O(n)		
Linked List	Insert at head	O(1)	O(1)	O(n)
	Search / Access	O(n)	O(n)	
BST (balanced)	Search / Insert	O(log n)	O(n)*	O(n)
	* worst = skewed tree			
Trie	Insert / Search	O(L)	O(L)	O(N·L)
Graph (adj list)	DFS / BFS	O(V+E)	O(V+E)	O(V+E)

6.2 Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Timsort (Python default)	O(n)	O(n log n)	O(n log n)	O(n)	Yes	list.sort(), sorted()
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Divide & conquer
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	In-place, pivot
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Uses heap
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Good for small n or nearly sorted
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes	Integer keys only

Python Tip	Python's list.sort() and sorted() use Timsort — O(n log n) worst, O(n) best on nearly sorted data. Always use the built-in unless the problem asks you to implement sorting yourself.

6.3 Algorithm Pattern Complexity

Pattern	Time	Space	Key Note
Two Pointers	O(n) or O(n log n)*	O(1)	* includes sort if needed
Sliding Window	O(n)	O(k)	k = window or alphabet size
Prefix Sum	O(n) build, O(1) query	O(n)	
Hash Map Counting	O(n)	O(n)	
Binary Search	O(log n)	O(1)	
Binary Search on Answer	O(n log R)	O(1)	R = answer range
Monotonic Stack	O(n)	O(n)	Each element pushed and popped once

Monotonic Queue	O(n)	O(k)	k = window size
Fast & Slow Pointers	O(n)	O(1)	
DFS (tree)	O(n)	O(h)	h = tree height
DFS (graph)	O(V+E)	O(V)	
BFS	O(V+E)	O(V)	
Backtracking (subsets)	O(2^n)	O(n)	
Backtracking (perms)	O(n!)	O(n)	
DP 1D	O(n)	O(n)	O(1) with rolling variable
DP 2D	O(m·n)	O(m·n)	O(n) with rolling row
Greedy (with sort)	O(n log n)	O(1)	
Union Find	O(α(n))	O(n)	α = inverse Ackermann ≈ O(1)
Topological Sort	O(V+E)	O(V+E)	
Dijkstra	O((V+E)logV)	O(V+E)	Min-heap required
Trie insert/search	O(L)	O(N·L)	L=word length, N=word count

6.4 Python Built-in Operations

Operation	Complexity	Note
len(x)	O(1)	Lists, dicts, sets, strings all cache length
list[i]	O(1)	Random access
list.append(x)	O(1) amort	Dynamic array
list.pop()	O(1)	From end
list.pop(0)	O(n)	□ Use deque.popleft() instead
list.insert(i, x)	O(n)	Shifts right of i
list.sort()	O(n log n)	Timsort, in-place
sorted(iterable)	O(n log n)	Returns new list
x in list	O(n)	Linear scan
x in set / dict	O(1)	Hash lookup
dict[k] = v / dict.get(k)	O(1)	
dict.items() iteration	O(n)	

set.add() / set.discard()	O(1)	
set union / intersection	O(m+n)	m, n = sizes of sets
heapq.heappush	O(log n)	
heapq.heappop	O(log n)	
heapq.heapify	O(n)	Better than n pushes
bisect.bisect_left	O(log n)	Binary search on sorted list
bisect.insort	O(n)	O(log n) search + O(n) insert shift
deque.appendleft / popleft	O(1)	list.pop(0) is O(n)!
Counter(iterable)	O(n)	
".join(list)	O(n)	Single allocation — always use this
str += str (in loop)	O(n^2) total	□ Creates new string every time

6.5 Complexity Decision Guide

n (input size)	Max complexity you can afford	Patterns that work
n ≤ 10	O(n!)	Any — brute force, all permutations
n ≤ 20	O(2^n)	Backtracking, bitmask DP
n ≤ 100	O(n^3)	3-nested loops, Floyd-Warshall, O(n^3) DP
n ≤ 1,000	O(n^2)	Nested loops, O(n^2) DP, insertion sort
n ≤ 100,000	O(n log n)	Sorting, Binary Search, Heap, Divide & Conquer
n ≤ 1,000,000	O(n)	Two Pointers, Sliding Window, Hash Map, Prefix Sum, Linear DP
n > 10,000,000	O(n) or O(log n)	Binary Search, Counting Sort, single-pass algorithms

6.6 Big-O Growth Rate Reference

Complexity	Name	n=10	n=100	n=1,000	n=1,000,000
O(1)	Constant	1	1	1	1
O(log n)	Logarithmic	3	7	10	20
O(n)	Linear	10	100	1,000	1,000,000
O(n log n)	Linearithmic	33	664	10,000	20,000,000
$O(n^2)$	Quadratic	100	10,000	1,000,000	10^{12} ✕
$O(n^3)$	Cubic	1,000	1,000,000	10^9 ✕	10^{18} ✕
O(2^n)	Exponential	1,024	10^{30} ✕	—	—
O(n!)	Factorial	3.6M	—	—	—

Interview Rule	If your solution is $O(n^2)$ or worse and n > 10,000 — you will almost certainly get TLE (Time Limit Exceeded). Always check constraints first and target the correct complexity before writing a single line of code.

6.7 Master Quick Reference

Problem type	Target complexity	Pattern / Tool
Pair in sorted array	O(n)	Two Pointers
Subarray sum / count subarrays	O(n)	Prefix Sum + Hash Map
Longest / shortest substring	O(n)	Sliding Window
Frequency / grouping / lookup	O(n)	Hash Map / Counter
Search in sorted data	O(log n)	Binary Search
Next greater / span	O(n)	Monotonic Stack
Sliding window max/min	O(n)	Monotonic Deque
Cycle or middle in linked list	O(n)	Fast & Slow Pointers
Connected regions on grid/graph	O(V+E)	DFS
Shortest path (unweighted)	O(V+E)	BFS
All combinations / subsets	O(2^n)	Backtracking
Minimum cost / count ways / optimal	O(n·states)	DP
Interval merge / scheduling	O(n log n)	Greedy + Sort
Group / union elements dynamically	O(α(n))	Union Find

Task ordering / prerequisites	O(V+E)	Topological Sort
Shortest path (weighted graph)	O((V+E)logV)	Dijkstra
Prefix word match / autocomplete	O(L)	Trie
Top K elements / K-th largest	O(n log k)	Heap

Book Complete. You now have everything needed to recognise patterns, apply templates, and solve most LeetCode interview problems efficiently. Revise Section 4 before every session. Revise Section 6 before every interview. Practice daily.